BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI — WORK INTEGRATED LEARNING PROGRAMMES

Heart Disease Risk Prediction: An End-to-End MLOps System

Design, implementation and verification of a reproducible, tested, containerised, orchestrated and monitored machine-learning service

Name	Snehanjali Gandham
BITS ID	2025AE05303
Email	2025ae05303@wilp.bits-pilani.ac.in
Course	MLOps (AIMLZG523) — Assignment 1
Code repository	https://github.com/SnehanjaliGandham/heart-disease-prediction-mlops (public)
Demo video	https://drive.google.com/file/d/1LBT4Bhkk0Xh9PKlxUtrgJyhp2BtB-PTL/view?usp=sharing
Container image	ghcr.io/snehanjaligandham/heart-disease-api:latest (built, tested and published by the CI pipeline)
API access	Local Kubernetes (Docker Desktop): POST http://localhost:8080/predict — setup in Section 6.1, access instructions in Section 9.1

Abstract

This report presents the design and implementation of a production-grade machine-learning system that estimates the risk of heart disease from thirteen routine clinical measurements, using the Cleveland subset of the UCI Heart Disease dataset [1]. Beyond training an accurate classifier, the work addresses the operational concerns that MLOps practice identifies as the main source of failure in deployed models [2]: data integrity, experiment traceability, reproducibility, automated verification, safe deployment and post-deployment monitoring. Three model families were tuned with stratified five-fold cross-validation and compared under a selection rule declared before training; Logistic Regression was selected with a cross-validated ROC-AUC of 0.911 ± 0.018 and achieved a ROC-AUC of 0.961 and a recall of 0.929 on 61 held-out patients. All runs are recorded in MLflow with a model registry. A five-stage GitHub Actions pipeline enforces 199 automated tests (99 % line coverage), a model quality gate, container verification and a Kubernetes deployment test on every change. Deployed on Kubernetes, the FastAPI service served 22,880 requests without error while autoscaling from two to five replicas, and its Prometheus/Grafana monitoring detected a deliberately shifted patient population within five minutes. The complete training pipeline was re-executed on the BITS Prayogshala virtual lab with identical results.

Keywords: *MLOps*, *reproducibility*, *experiment tracking*, *CI/CD*, *Docker*, *Kubernetes*, *autoscaling*, *monitoring*, *data drift*, *heart disease*.

1. Introduction

A model that performs well in a notebook is only the starting point of a useful system. Surveys of MLOps practice [2] observe that most of the effort, and most of the risk, lies in the surrounding infrastructure: how data is obtained and checked, how experiments are compared, how a model is packaged, tested and released, and how it is observed once real traffic arrives. The objective of this assignment is therefore to build the complete life-cycle around a clinically motivated classifier and to demonstrate each stage with evidence rather than description.

Contributions. (i) A verified, leak-free data pipeline with documented exploratory analysis; (ii) a transparent model-selection protocol with an over-fitting check and a tested engineered feature; (iii) full experiment tracking and model registration; (iv) a packaged model that reproduces identically on three independent machines; (v) a five-stage CI/CD pipeline with 199 tests that demonstrably blocks faulty changes; (vi) a hardened container and a Kubernetes deployment verified under load, failure and update; and (vii) monitoring with tested alert rules and working data-drift detection. Table 1 maps each required deliverable to its location in the repository; all 80 screenshots are indexed by task in `screenshots/README.md`.

Table 1. *Assignment deliverables and their location in the repository.*

Deliverable	Location
Source code, Dockerfile, requirements	`src/`, `Dockerfile`, `requirements.txt` (version ranges), `requirements-lock.txt` (exact versions), `requirements-serve.txt` (API image)
Cleaned dataset and download script	`data/heart_disease_clean.csv`, `scripts/download_data.py`, SHA-256 manifest
Notebooks and scripts	`notebooks/01–03` (executed), `scripts/run_pipeline.sh` and `.ps1` (complete pipeline)
Unit tests and CI/CD workflow	`tests/` (199 tests), `.github/workflows/ci.yml` (five jobs)
Deployment and monitoring	`deployment/k8s/`, `deployment/monitoring/`, `deployment/k8s-optional/`, `docker-compose.yml`

2. System architecture

Figure 1 shows the three parts of the system. The **training plane** turns one verified data file into a packaged model and an MLflow record; it runs identically on the development laptop, on the BITS Prayogshala lab and in CI, which is what makes the results reproducible. The **delivery plane** is the GitHub Actions pipeline: every push is linted, tested, retrained, packaged into a container image, and deployed to a temporary Kubernetes cluster before it can be merged. The **serving plane** runs that image on Kubernetes behind a load balancer and an autoscaler, while Prometheus scrapes every replica and Grafana presents service health, model behaviour and data drift.

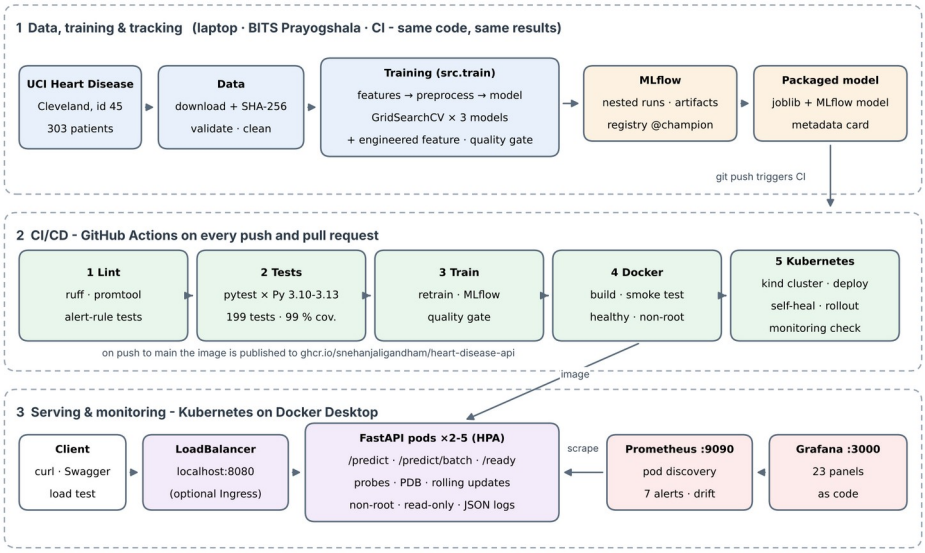


Figure 1. *System architecture: training plane (top), delivery plane (middle) and serving and monitoring plane (bottom).*

2.1 Technology selection

Each tool was chosen for a specific engineering property; the alternatives considered are listed so that the trade-off is explicit.

Table 2. *Technology choices, their justification and the alternatives considered.*

Concern	Choice	Justification	Alternative considered
Modelling	scikit-learn Pipeline [3]	All preprocessing and the estimator form one object, re-fitted inside every cross-validation fold (preventing leakage) and executed identically at serving time.	Separate preprocessing code — risks training/serving skew
Experiment tracking	MLflow 3 with SQLite [4]	Nested runs, artifacts and a model registry with aliases in a single local tool.	Weights & Biases — requires a hosted account
Serving	FastAPI with Uvicorn	Typed request schemas provide automatic input validation (HTTP 422) and interactive documentation.	Flask — validation must be hand-written
CI/CD	GitHub Actions	Integrated with the repository, no server to maintain, artifacts and summaries per run.	Jenkins — self-hosted server
Containerisation	Docker, two-stage build	Runtime layer contains serving dependencies only (154 MB), runs as a non-root user, includes a health check.	Single-stage image — larger, ships build tools
Orchestration	Kubernetes with Kustomize [5]	Self-healing, rolling updates and autoscaling; the manifests run unchanged on managed cloud clusters.	Helm — additional tooling; cloud cluster — cost
Monitoring	Prometheus and Grafana	Kubernetes service discovery scrapes every replica automatically; alert rules are unit-testable; dashboards are provisioned as code.	Log inspection only — no time series or alerting

Table 3. *Execution environments.*

Environment	Specification	Purpose
Development laptop	Windows 11, Intel i7, 16 GB RAM, Python 3.13, Docker Desktop 4.95 with Kubernetes v1.36	Development, training, container and Kubernetes experiments, monitoring
BITS Prayogshala	Linux virtual lab, Python 3.12.9	Independent re-execution of data, EDA, training, MLflow, tests and the complete pipeline
GitHub-hosted runner	ubuntu-24.04	All five CI/CD jobs on every push, including a temporary kind cluster

3. Data acquisition and exploratory analysis

3.1 Dataset

The study uses the Cleveland subset of the UCI Heart Disease collection (dataset 45, licence CC BY 4.0) [1], the data analysed in the original Cleveland Clinic study [6]. It contains 303 patients described by 13 clinical attributes — demographics (age, sex), symptoms (chest-pain type, exercise-induced angina), resting measurements (blood pressure, cholesterol, fasting blood sugar, ECG) and exercise-test results (maximum heart rate, ST depression and slope, number of major vessels, thallium scan). The diagnosis `num` ranges from 0 to 4 and was converted into a binary target: 0 for no disease and 1 for any degree of disease. A widely circulated 1,025-row version of this dataset was deliberately not used, because most of its rows are duplicates of the same patients, which would inflate every reported score.

3.2 Acquisition, validation and cleaning

`scripts/download_data.py` retrieves the data through UCI's official `ucimlrepo` client, verifies its shape and records a SHA-256 checksum in a manifest. Every later run accepts the local copy only if the checksum matches, so all experiments are guaranteed to use the same bytes; the checksum is computed on line-ending-normalised content so that Windows and Linux agree. `src/data.py` then validates the schema and data types, rejects category codes that UCI does not define and numeric values outside clinically plausible ranges, and removes duplicate rows. On the real file, no row was invalid or duplicated; six values were missing (`ca`: 4, `thal`: 2); and the classes are nearly balanced at 164 healthy and 139 diseased patients (45.9 % positive). These gaps are intentionally left in the cleaned file: imputation happens inside the model pipeline using statistics learned from training folds only, so that no information from validation or test patients leaks into training.

3.3 Exploratory data analysis

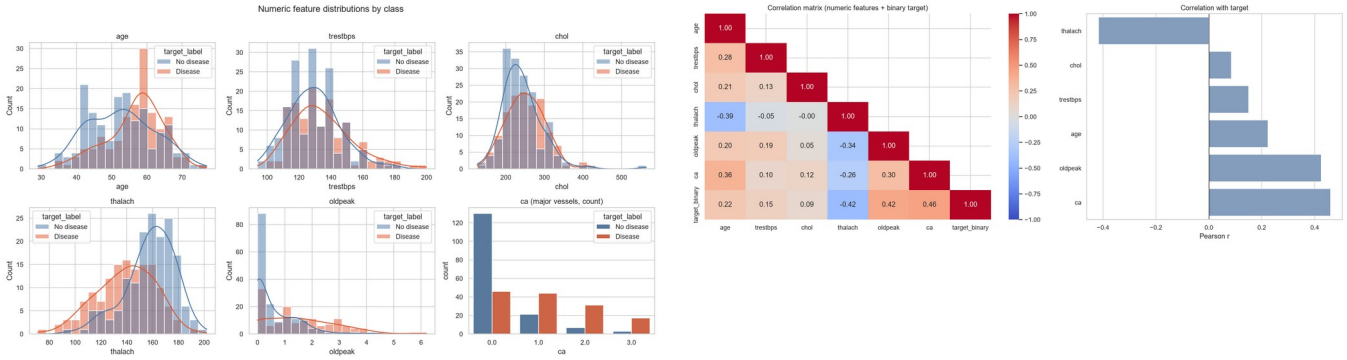


Figure 2. *Distributions of the numeric attributes by diagnosis (left) and Pearson correlation matrix including the target (right).*

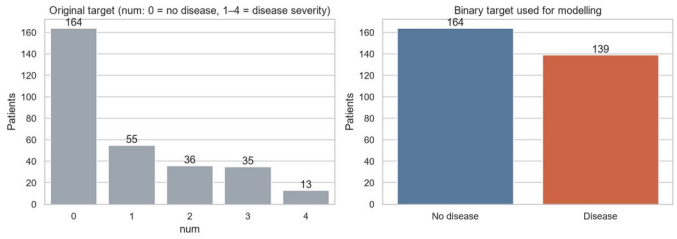


Figure 3. *Class balance after binarisation: 164 patients without and 139 with heart disease (45.9 % positive).*

Table 4. *Principal findings of the exploratory analysis and their consequence for modelling.*

Finding	Consequence
The strongest associations with the target are thal (r = 0.53), ca (0.46), exang (0.43), oldpeak (0.42), cp (0.41) and thalach (−0.42); cholesterol (0.09) and fasting blood sugar (0.03) are weak predictors on their own.	A linear model is a strong candidate. All attributes are retained and regularisation decides their weight.
Patients with disease are older (mean 56.6 vs 52.6 years), reach a lower maximum heart rate (139 vs 158 bpm) and show more ST depression (1.57 vs 0.59) and more blocked vessels (1.14 vs 0.27).	Motivates an age-adjusted heart-rate feature (Section 4.2).
Several attributes are skewed (cholesterol up to 564 mg/dl) but physiologically plausible; cp, thal, slope and restecg are category codes rather than quantities.	Outliers are kept; median imputation and standardisation for numeric attributes; one-hot encoding for codes.

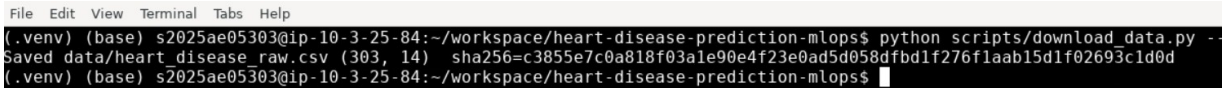


Figure 4. *Data acquisition re-executed on BITS Prayogshala: the downloaded file matches the recorded SHA-256 checksum.*

4. Feature engineering and model development

4.1 Preprocessing

Numeric attributes are imputed with the median and standardised; categorical attributes are imputed with the most frequent value and one-hot encoded, ignoring categories not seen during training so that the API cannot fail on an unexpected code. The result is 25 model features. Because these steps are part of the scikit-learn Pipeline [3], they are re-fitted inside every cross-validation fold.

4.2 Engineered feature

Exercise physiology expresses a patient's heart-rate response as the share of the age-predicted maximum (220 − age) that is actually reached; a low share, known as chronotropic incompetence, is an established marker of coronary disease [7]. The custom transformer `ClinicalFeatures` adds this quantity as `hr_pct_predicted = thalach / (220 − age)`. In this dataset its mean is 0.85 for patients with disease and 0.95 for those without. Rather than assuming that the feature helps, it was exposed to the hyper-parameter search as an on/off switch, so its inclusion is decided by cross-validated evidence.

4.3 Candidate models, tuning and selection protocol

The data was split once, stratified by diagnosis, into 242 development patients and a held-out test set of 61 patients (random seed 42). Each model family was tuned with `GridSearchCV` using stratified, shuffled five-fold cross-validation [8], recording accuracy, precision, recall, F1 and ROC-AUC and refitting on ROC-AUC.

Table 5. *Candidate models and hyper-parameter search spaces (153 configurations, 765 fits in total).*

Model	Hyper-parameters searched (each grid also includes the engineered-feature switch)	Configs
Majority-class baseline	None — always predicts the most frequent class; establishes the lower bound	1
Logistic Regression	C ∈ {0.01, 0.1, 1, 10}; class weight ∈ {none, balanced}; L1 ratio ∈ {0 (L2), 0.5, 1 (L1)}	48
Random Forest	trees ∈ {200, 500}; max depth ∈ {none, 4, 8}; min samples per leaf ∈ {1, 3, 5}; max features ∈ {sqrt, 0.5}	72
Gradient Boosting	stages ∈ {100, 200}; learning rate ∈ {0.03, 0.1}; max depth ∈ {2, 3}; subsample ∈ {0.8, 1.0}	32

Selection protocol. To avoid choosing a model on the basis of noise, the rule was fixed in `src/config.py` before training: models whose mean cross-validated ROC-AUC lies within 0.01 of the best are considered equivalent, and the simplest of them is selected (Logistic Regression, then Random Forest, then Gradient Boosting). The test set plays no part in selection. Over-fitting is quantified for each model as the difference between its mean training-fold and validation-fold ROC-AUC.

4.4 Results

Table 6. *Cross-validated performance on the development data and final performance on the 61 held-out test patients.*

Model	CV ROC-AUC	Over-fit gap	Accuracy	Precision	Recall	Test ROC-AUC
Logistic Regression (selected)	0.911 ± 0.018	0.015	0.885	0.839	0.929	0.961
Random Forest	0.900 ± 0.032	0.057	0.885	0.839	0.929	0.955
Gradient Boosting	0.884 ± 0.027	0.077	0.902	0.867	0.929	0.960
Majority-class baseline	0.500	0.000	0.541	0.000	0.000	0.500

Logistic Regression (L2 penalty, C = 0.1) satisfies every pre-declared criterion: it has the highest and most stable cross-validated ROC-AUC, and its over-fitting gap is four to five times smaller than that of the tree ensembles. It is also the most interpretable and the cheapest model to serve. On the test set it identifies 26 of the 28 patients with disease. Gradient Boosting's slightly higher test accuracy corresponds to a single patient out of 61 and does not affect the decision, since the test set is reserved for the final estimate. Test scores above the cross-validated estimate are consistent with sampling variation in a small test set. The engineered feature did not improve any model (best CV ROC-AUC without / with: LR 0.9113 / 0.9090, RF 0.8998 / 0.8985, GB 0.8839 / 0.8835) — once age and maximum heart rate are both available, their ratio carries no additional information. This negative result is reported as such, and the feature remains available as a tested option.

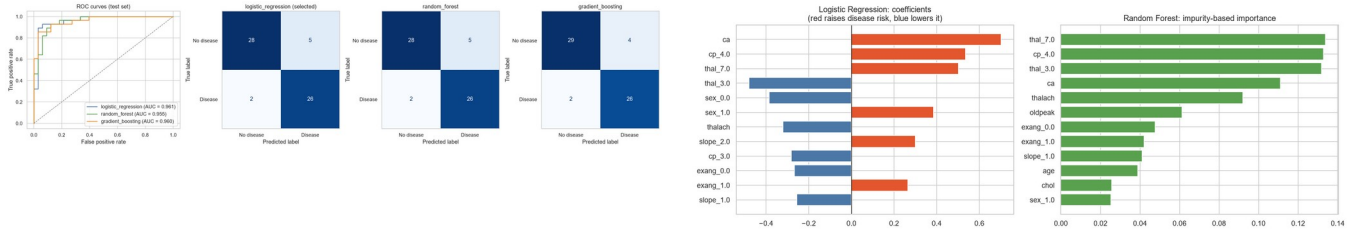


Figure 5. *Selected model on the test set: ROC curve and confusion matrix (left). Logistic Regression coefficients and Random Forest importances agree on the leading predictors ca, thal, cp and thalach (right).*

5. Experiment tracking

Reproducible research requires that every result can be traced back to the code, data and settings that produced it. All training runs are therefore recorded in the MLflow [4] experiment `heart-disease-classification`, organised as summarised in Table 7.

Table 7. *Information recorded by MLflow at each level.*

Level	Recorded information
Parent run (one per training execution)	Git commit, Python and library versions; split ratio, number of folds, random seed, data SHA-256 and the selection rule; the training dataset registered as an MLflow input; the model-comparison table, feature-ablation table, ROC curves and the written reason for the selection.
Child run (one per model family)	Best hyper-parameters; mean and standard deviation of five cross-validated metrics; test-set metrics; the full cross-validation results; confusion-matrix and feature-importance plots; the fitted pipeline as an MLflow model with input signature, example input and its source files.
Model registry	The selected model is registered as `heart-disease-classifier`, and the alias `@champion` is moved to the new version, identifying the model that should be deployed.

A unit test asserts that the metrics stored in MLflow are identical to those returned by the training code, so the tracking record cannot silently diverge from the actual results.

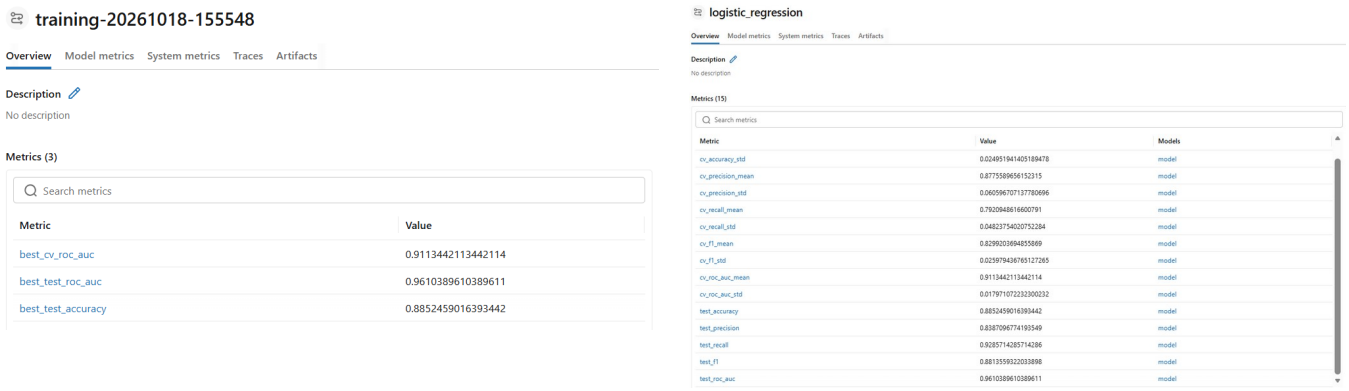


Figure 6. *MLflow on the development laptop: the parent run with its summary metrics (left) and the nested Logistic Regression run with all fifteen logged metrics (right).*

Lineage across environments. On BITS Prayogshala, three training executions — one from the command line and two from notebook 02 — produced fifteen runs. MLflow records the source of each run (`train.py` or the notebook) and the dataset digest `0cc22ed3` for every run, which demonstrates that all models were trained on identical data (Figure 7). Each execution registered a new model version, and the `@champion` alias moved to the most recent one without manual intervention (Figure 8, right).

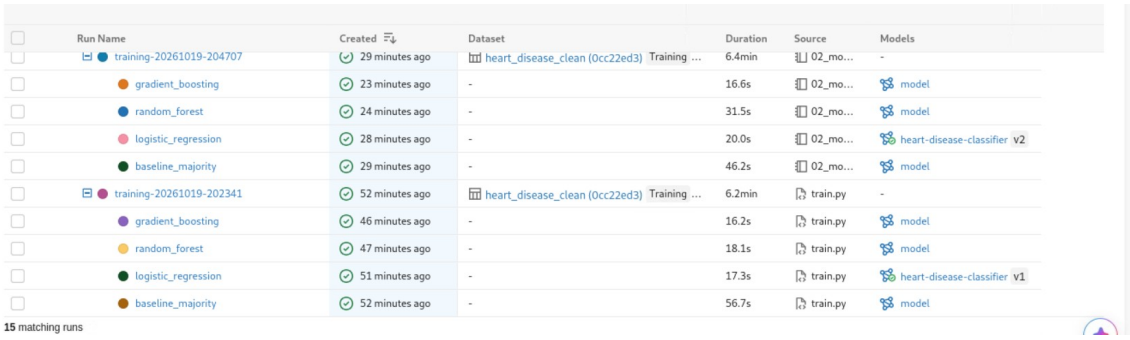


Figure 7. *MLflow on BITS Prayogshala: two of the three training executions, each a parent run with four nested model runs, showing the source, the shared dataset digest and the registry version linked to each Logistic Regression run.*

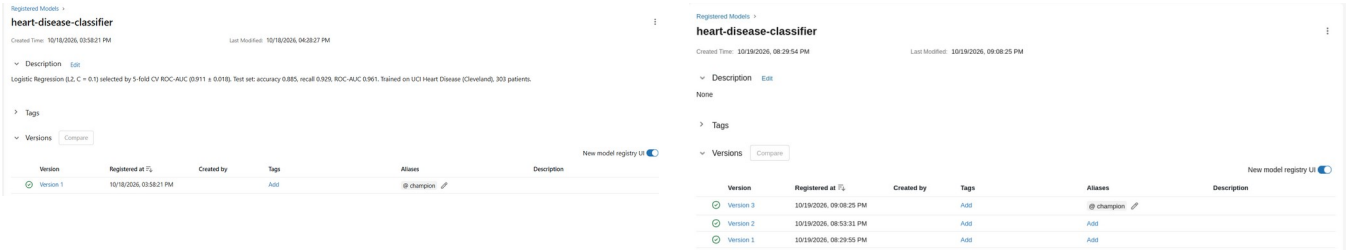


Figure 8. *Model registry: heart-disease-classifier with the @champion alias on the laptop (left) and versions 1–3 on BITS Prayogshala (right).*

6. Model packaging and reproducibility

The final model is saved in three complementary forms so that it can be served, re-loaded and audited:

Table 8. *Packaged model artifacts in the `models/` directory.*

Artifact	Purpose
`heart_disease_model.joblib`	The complete fitted pipeline — engineered feature, preprocessing and classifier. Callers supply the 13 raw attributes; no separate preprocessing has to be kept in step.
`mlflow_model/`	The same pipeline in MLflow format, bundling the project source files it imports, so it can be loaded outside this repository.
`model_metadata.json`	A model card: hyper-parameters, cross-validated and test metrics, data checksum, git commit, training timestamp and library versions. The API exposes it at `/model-info`.

Safeguards. Immediately after saving, the training script reloads the file and asserts that it returns exactly the same probabilities as the in-memory model. When the API loads a model, it compares the installed scikit-learn version with the one recorded at training time and warns on a mismatch. Finally, `scripts/quality_gate.py` rejects any model whose cross-validated or test ROC-AUC falls below 0.85, whose recall falls below 0.80, whose accuracy falls below 0.75, or whose cross-validated ROC-AUC exceeds the baseline by less than 0.2; in CI this failure stops the pipeline.

Environment control. `requirements.txt` specifies compatible version ranges (Python 3.9–3.13), `requirements-lock.txt` records the exact versions used for the reported results, random seeds are fixed for the split, the folds and every model, and `.gitattributes` normalises line endings. The whole pipeline — environment check, download, cleaning, training with MLflow, quality gate, sample prediction, linting and tests — runs with a single command, `bash scripts/run_pipeline.sh` (or `scripts\run_pipeline.ps1` on Windows), and stops at the first failure.

6.1 Setup and installation

```
git clone https://github.com/SnehanjaliGandham/heart-disease-prediction-mlops.git && cd heart-disease-prediction-mlops
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt                         # Python 3.9-3.13
bash scripts/run_pipeline.sh                            # Windows: .\scripts\run_pipeline.ps1
mlflow ui --backend-store-uri sqlite:///mlflow.db       # experiment browser on port 5000
```

Independent verification. Table 9 compares three independent executions. The scores agree to the fourth decimal place across two operating systems and two Python versions, which confirms that the pipeline is reproducible from the repository alone.

Table 9. *Cross-validated and test ROC-AUC obtained on three independent machines.*

Execution environment	LR (CV)	RF (CV)	GB (CV)	LR (test)
Development laptop — Windows 11, Python 3.13	0.9113	0.8998	0.8839	0.961
GitHub Actions — ubuntu-24.04, Python 3.12 (run #7)	0.9113	0.8998	0.8839	0.961
BITS Prayogshala — Linux, Python 3.12.9	0.9113	0.8998	0.8839	0.961

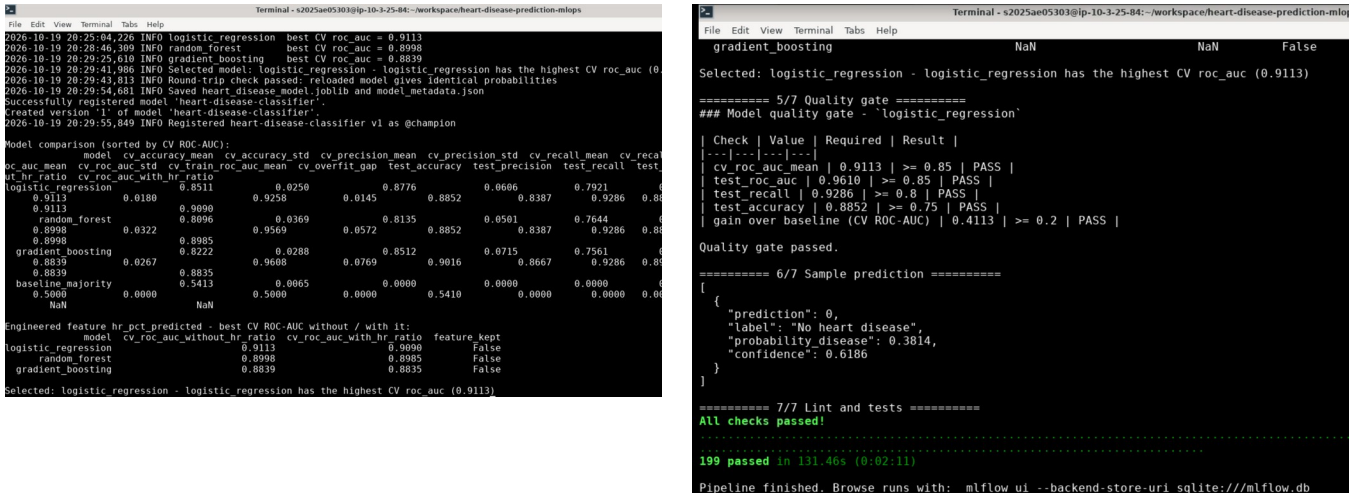


Figure 9. *BITS Prayogshala (user s2025ae05303). Left: python -m src.train — identical cross-validated scores, the feature-ablation table and the logged selection reason. Right: the one-command pipeline — quality gate passed, sample prediction, clean lint and 199 passing tests.*

7. CI/CD pipeline and automated testing

7.1 Test strategy

Following the ML Test Score rubric [9], the test suite covers not only code paths but also the data, the model and the infrastructure. It consists of 199 Pytest tests with 99 % line coverage; the tests run offline and deterministically, and those involving training use a small but real grid with real MLflow logging so that they finish in seconds. The same suite was also executed on BITS Prayogshala, where all 199 tests passed with 99 % coverage (Figure 9, right).

Table 10. *Scope of the automated test suite.*

Area	What the tests verify
Data	checksum (incl. Windows line endings), schema, invalid codes and ranges, duplicates, target mapping, stratified split
Features and models	no missing values after transformation, no leakage across folds, engineered-feature formula and switch, every grid
Training and tracking	full training run, selection rule, over-fitting gap, ablation, MLflow records equal returned metrics, reload equality
Serving	API contract, ten invalid-input cases (HTTP 422), batch limits, request IDs, no patient data in logs, HTTP 503 before load
Infrastructure	probes target real routes, ports match the image, selectors and security settings, every monitored metric exists

7.2 Pipeline

The workflow `.github/workflows/ci.yml` runs on every push and pull request. Its five jobs depend on one another, so a failure at any stage prevents the later stages from running; each job uploads its logs and reports as artifacts.

Table 11. *Jobs of the CI/CD pipeline.*

Job	Steps	Fails when
1. Static checks	ruff linting; promtool validation of the Prometheus configuration and unit tests of the alert rules	lint error, invalid configuration, alert behaves incorrectly
2. Tests	Pytest with coverage on Python 3.10, 3.11, 3.12 and 3.13; JUnit and HTML reports	any failing test or coverage below 90 %
3. Model	verify data checksum → clean → train with MLflow → quality gate → sample inference → model tests → execute notebooks	any error or a failed quality gate
4. Container	build with the job-3 model → run → smoke test → wait for healthy → verify non-root user → publish to GHCR	build, smoke-test or health-check failure
5. Kubernetes	create a kind cluster → deploy → smoke test through the Service → delete a pod → rolling update → deploy monitoring → verify scraping	pods not ready or any verification fails

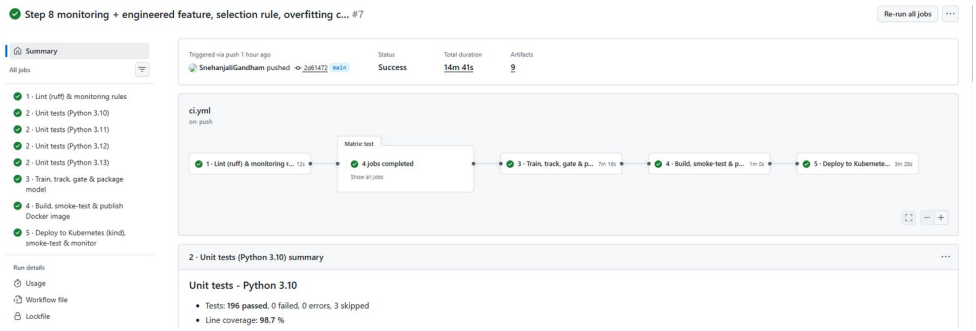


Figure 10. *A complete pipeline run on the main branch: all five jobs succeed in 14 min 41 s, nine artifacts are uploaded, and the job summary reports 196 tests passed on Python 3.10 with 98.7 % coverage.*

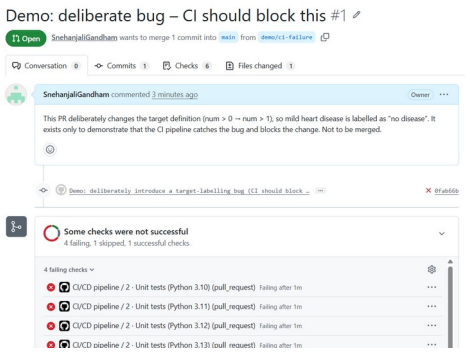


Figure 11. *A pull request containing a deliberately failing test: the test jobs fail on all four Python versions and the later jobs never run, so the change cannot be merged.*

8. Model containerisation and prediction API

Image design. The `Dockerfile` uses two stages based on `python:3.13-slim`. The runtime stage installs only the serving dependencies listed in `requirements-serve.txt` — MLflow, Jupyter and plotting libraries are excluded — which keeps the compressed image at 154 MB and reduces its attack surface. Code and model are copied read-only, the process runs as the unprivileged user 10001, and a Docker `HEALTHCHECK` queries the readiness endpoint. The CI pipeline publishes each build of the main branch to the GitHub Container Registry, tagged with the commit hash and `latest`.

Table 12. *Endpoints of the prediction API.*

Endpoint	Behaviour
POST /predict	Accepts one patient (13 attributes as JSON) and returns the predicted class, label, probability of disease, confidence, model version and a request identifier.
POST /predict/batch	Accepts up to 100 patients in a single request.
GET /health	Liveness: returns 200 while the process is running.
GET /ready	Readiness: returns 200 once the model is loaded, otherwise 503.
GET /model-info, /metrics, /docs	Model card and serving pod; Prometheus metrics; interactive Swagger documentation.

Input validation and error handling. Every numeric attribute is checked against a clinically plausible range and every coded attribute against the codes defined by UCI; `ca` and `thal` may be null because the pipeline imputes them, and unknown fields are refused. Invalid requests receive HTTP 422 with one message per offending field, and unexpected errors produce a generic HTTP 500 response — a stack trace is never returned. An example exchange:

```
POST /predict  {"age":67,"sex":1,"cp":4,"trestbps":160,"chol":286,"fbs":0,"restecg":2,"thalach":108,
                "exang":1,"oldpeak":1.5,"slope":2,"ca":3,"thal":3}
200 OK  {"prediction":1,"label":"Heart disease","probability_disease":0.9787,"confidence":0.9787,
         "model_version":"logistic_regression-c19ff10","request_id":"smoke-test"}
```



Figure 12. *scripts/smoke_test.py passes all eight end-to-end checks against the running container (left); a prediction made through the Swagger interface (right).*

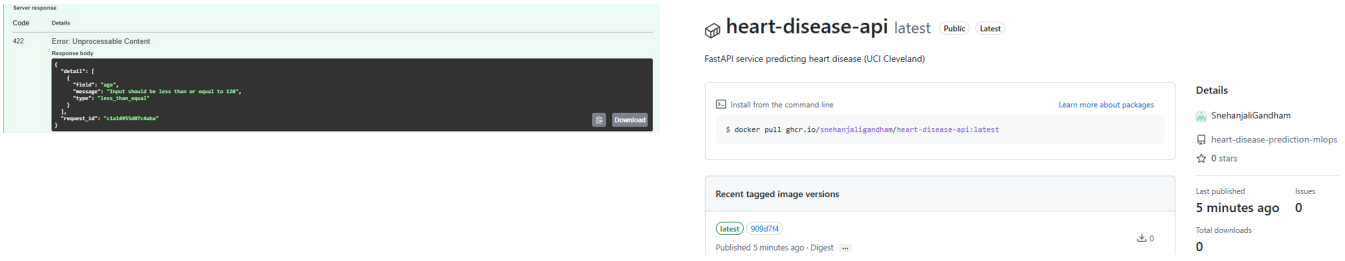


Figure 13. *An out-of-range request rejected with HTTP 422 and a message per field (left); the image published to the GitHub Container Registry by CI (right).*

9. Production deployment on Kubernetes

The service is deployed to the Kubernetes cluster included in Docker Desktop, which the assignment permits; because the manifests contain nothing specific to that cluster, they apply unchanged to managed services such as GKE, EKS or AKS [5]. A single command, `kubectl apply -k deployment/k8s`, creates the resources in Table 13.

9.1 Access instructions (local testing)

```
docker build -t heart-disease-api:1.0.0 .                          # image used by the cluster
kubectl apply -k deployment/metrics-server && kubectl apply -k deployment/k8s
python scripts/smoke_test.py --url http://localhost:8080            # 8 end-to-end checks
curl -X POST http://localhost:8080/predict -H "Content-Type: application/json" -d @samples/high_risk.json
# Swagger UI: http://localhost:8080/docs   ·   monitoring: kubectl apply -k deployment/monitoring
```

Table 13. *Kubernetes resources and the production concern each one addresses.*

Resource	Configuration and purpose
Deployment	Two replicas; rolling updates with surge 1 and unavailability 0, plus a 5-second pre-stop pause, so capacity never drops during an update; startup and readiness probes on /ready and a liveness probe on /health; CPU and memory requests and limits; a hardened security context (non-root user, read-only root filesystem, all Linux capabilities dropped, seccomp profile, no service-account token).
Service (LoadBalancer)	A single stable address on port 8080 that distributes traffic across ready replicas; a cloud provider would assign a public IP. An Ingress alternative is provided in deployment/k8s-optional/.
HorizontalPodAutoscaler	Scales between two and five replicas to keep average CPU utilisation near 60 % (metrics-server is installed from deployment/metrics-server/).
PodDisruptionBudget, ConfigMap	At least one replica remains available during voluntary disruptions; the decision threshold and log level can be changed without rebuilding the image.

Each production property was verified experimentally rather than assumed (Table 14).

Table 14. *Operational experiments on the deployed service.*

Experiment	Observed result
Deployment and smoke test through localhost:8080	Both replicas ready in about 11 s; all 8 checks pass; requests distributed evenly (5 / 5) between replicas.
Deletion of a running pod	A replacement started within seconds; no request failed.
Load test: 16 concurrent clients for 3 minutes	CPU rose from 5 % to 791 % of the request; the autoscaler scaled 2 → 5 → 2 replicas; 22,880 requests with 0 errors; median latency 100 ms, 95th percentile 300 ms.
Rolling update during load	Every replica replaced; 4,002 requests with 0 errors.
Faulty release followed by `kubectl rollout undo`	The new pod could not start (ImagePullBackOff) while the existing replicas continued to serve; the previous version was restored.



Figure 14. *Deployed resources: replicas, LoadBalancer service and autoscaler (left); load-test summary showing 22,880 requests without error (right).*

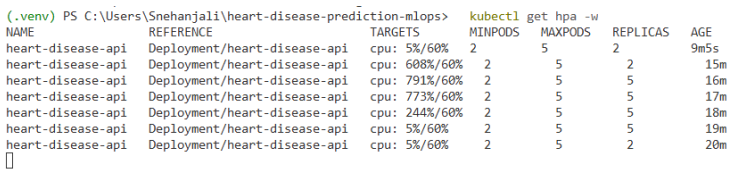


Figure 15. *Autoscaler behaviour under load: CPU utilisation rises to 791 % of the request, the deployment scales from two to five replicas and returns to two when traffic stops.*

10. Monitoring, logging and drift detection

Logging. The API writes one structured JSON record per request (timestamp, pod, request identifier, route, status and latency) and one per prediction. The request identifier is returned to the caller so that an individual prediction can be traced; patient attributes are never written to the logs, which is enforced by a unit test. **Metrics.** Every replica exposes request counts, a latency histogram, predictions per class, the distribution of predicted probabilities, rejected requests, summaries of each numeric input, the model version, CPU and memory use, and the training mean and standard deviation of each feature, read from the scaler of the model actually being served. Prometheus discovers the replicas through the Kubernetes API, so replicas added by the autoscaler are monitored automatically, and Grafana is provisioned entirely from configuration files. **Alerting.** Seven alert rules cover unavailable replicas, missing targets, server errors above 5 %, 95th-percentile latency above 500 ms, input drift, a shift in the share of positive predictions and a high rate of invalid requests; five of them are unit-tested with promtool in CI.

Drift detection. Because confirmed diagnoses arrive long after predictions, the service monitors its inputs, a recommended practice for detecting dataset shift [10]. For each feature, the drift score is the difference between the mean of the inputs received in the last 15 minutes and the training mean, divided by the training standard deviation; an alert fires when its magnitude exceeds 0.5 for five minutes. Under realistic traffic the scores remained within ±0.07. When an older, higher-risk population was sent, the scores reached +1.67 for age, −1.12 for maximum heart rate and +1.06 for cholesterol, the share of positive predictions rose from 43 % to 57 %, and the alert HeartApiInputDrift fired.



Figure 16. *Grafana drift panels during the population shift: every monitored feature leaves the ±0.5 band (left); recent versus training means, with age at +1.67 standard deviations (right).*



Figure 17. *Prometheus: HeartApiInputDrift firing for age and maximum heart rate (thalach); chol, oldpeak and trestbps pending.*

11. Discussion and conclusion

Problems encountered. Development exposed several realistic failure modes, each of which was fixed and protected by a check: Windows line endings altered the data checksum (resolved by normalised hashing); eight requests failed during an early rolling update because pods terminated before leaving the load balancer (resolved by a pre-stop pause, after which 4,002 requests passed without error); a model trained with a newer scikit-learn could not be loaded on Python 3.10 (resolved by a version check and locked-version tests); and the Prayogshala run revealed suppressed notebook charts and a pipeline script that ignored a lint failure (both corrected and re-verified). **Limitations.** The dataset is small and comes from a single hospital; with only 61 test patients, the test metrics carry wide confidence intervals; the decision threshold of 0.5 is not calibrated to clinical costs; and the cluster is local rather than publicly reachable. **Conclusion.** All tasks of the assignment have been implemented, executed and evidenced, and the resulting system exhibits the properties expected of a production service: it is reproducible, tested, self-healing, scalable and observable.

References

[1] A. Janosi, W. Steinbrunn, M. Pfisterer and R. Detrano, “Heart Disease,” UCI Machine Learning Repository, 1989. doi:10.24432/C52P4X.
[2] D. Kreuzberger, N. Kühl and S. Hirschl, “Machine Learning Operations (MLOps): Overview, Definition, and Architecture,” IEEE Access, vol. 11, pp. 31866–31879, 2023.
[3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
[4] M. Zaharia et al., “Accelerating the Machine Learning Lifecycle with MLflow,” IEEE Data Engineering Bulletin, vol. 41, no. 4, pp. 39–45, 2018.
[5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, “Borg, Omega, and Kubernetes,” ACM Queue, vol. 14, no. 1, pp. 70–93, 2016.
[6] R. Detrano et al., “International application of a new probability algorithm for the diagnosis of coronary artery disease,” American Journal of Cardiology, vol. 64, no. 5, pp. 304–310, 1989.
[7] P. H. Brubaker and D. W. Kitzman, “Chronotropic incompetence: causes, consequences, and management,” Circulation, vol. 123, no. 9, pp. 1010–1020, 2011.
[8] T. Hastie, R. Tibshirani and J. Friedman, The Elements of Statistical Learning, 2nd ed., ch. 7. Springer, 2009.
[9] E. Breck, S. Cai, E. Nielsen, M. Salib and D. Sculley, “The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction,” in Proc. IEEE Int. Conf. Big Data, 2017, pp. 1123–1132.
[10] S. Rabanser, S. Günnemann and Z. C. Lipton, “Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift,” in Advances in Neural Information Processing Systems 32, 2019.